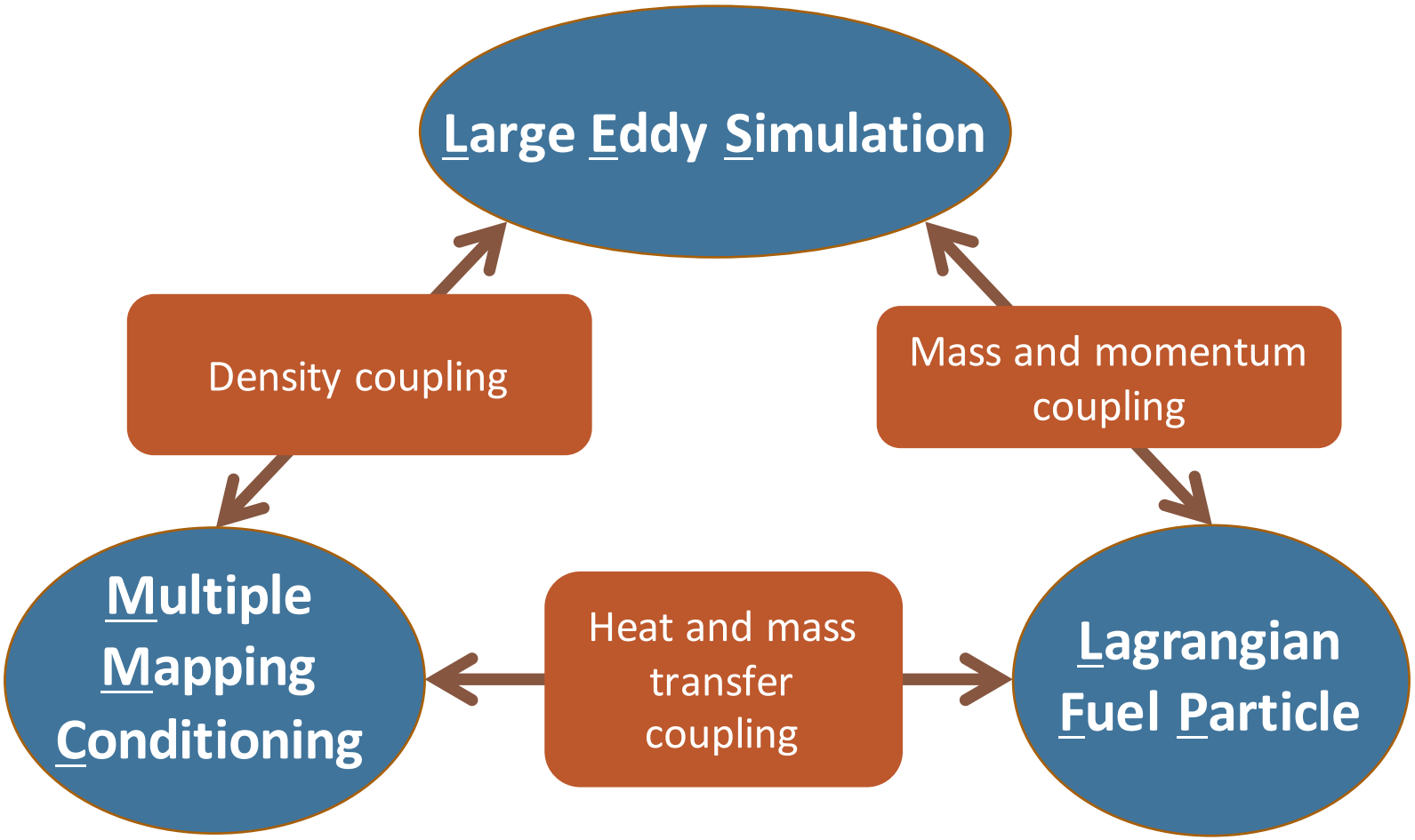
Documentation for:

A Mixture Fraction Based Model for the Turbulent Combustion of Diluted Solid Fuel Dispersions (Code)

Longfei ZHAO, PhD Student
Supervisor: Dr. Matthew Cleary
Associate Supervisor: Prof. Assaad Masri

May, 2017

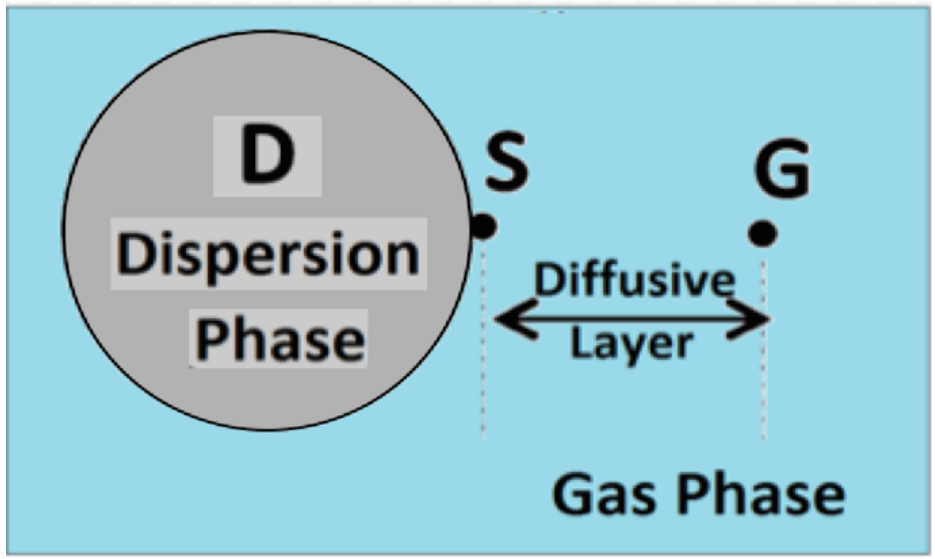
System Overview



Schematics of models and their interactions

Lagrangian Fuel Particle

❖ **Phase and State Definition**



u_D -- velocity of dispersed (D) state

h_S -- standardized enthalpy of surface (S) state

T_G -- temperature of remote (G) state

- ❖ Gas Phase
- ❖ Dispersed Phase
- ❖ For one single fuel particle
  - **D** ➔ Dispersed **state** (from dispersed **phase**)
  - **S** ➔ Surface **state** (interface of two **phases**)
  - **G** ➔ Remote **state** (from gaseous **phase**)

Solver – mmcMultiphaseFoam

```
while (runTime.run())
{

    #include "readTimeControls.H"
    #include "compressibleCourantNo.H"
    #include "setDeltaT.H"

    runTime++;
    Info<< "Time = " << runTime.timeName() << nl << endl;

    #include "rhoEqn.H"

    // --- PIMPLE loop
    while (pimple.loop())
    {
        #include "UEqn.H"

        D = turbulence->nu()/Sc_;
        Dt = turbulence->nut()/Sct_;
        DEff = D + Dt;

        #include "XiEqn.H"
        #include "hYEqvE_Eqn.H"

        // --- Pressure corrector loop
        while (pimple.correct())
        {
            #include "pEqn.H"
        }

        if (pimple.turbCorr())
        {
            turbulence->correct();
        }
    }
```

- Continuity Equation
- Momentum Equation
- MMC Coupling/Reference Variable Equation
- Primary Species Mass Fraction Equation

```
    rho = thermo.rho();

    gradRho = fvc::grad(rho*DEff);
    #include "checkParticles.H"    // only works with basicThermoCloud.
    #include "moveParticles.H"
    #include "sampleParticles.H"

    #include "moveFuelParticles.H"

    runTime.write();
}

Info<< "End mmcMultiphaseFoam \n" << endl;

return 0;
```

Solver – mmcMultiphaseFoam (Continuity)

```
while (runTime.run())
{

    #include "readTimeControls.H"
    #include "compressibleCourantNo.H"
    #include "setDeltaT.H"

    runTime++;
    Info<< "Time = " << runTime.timeName() << nl <<

    #include "rhoEqn.H"

    // --- PIMPLE loop
    while (pimple.loop())
    {
        #include "UEqn.H"

        D = turbulence->nu()/Sc_;
        Dt = turbulence->nut()/Sct_;
        DEff = D + Dt;

        #include "XiEqn.H"
        #include "hYEqvE_Eqn.H"

        // --- Pressure corrector loop
        while (pimple.correct())
        {
            #include "pEqn.H"
        }

        if (pimple.turbCorr())
        {
            turbulence->correct();
        }
    }
```

$$\frac{\partial \bar{\rho}}{\partial t} + \frac{\partial \bar{\rho}\tilde{u}_i}{\partial x_i} = \tilde{\dot{S}}_\rho$$

```
fvScalarMatrix rhoEqn
(
    fvm::ddt(rho)
  + fvc::div(phi)
  ==
    fvOptions(rho)
  + fuelCloud.SM(rho)
);

fvOptions.constrain(rhoEqn);

rhoEqn.solve();

fvOptions.correct(rho);
```

```

    rho = thermo.rho();

    gradRho = fvc::grad(rho*DEff);
    #include "checkParticles.H"    // only works with basicThermoCloud.
    #include "moveParticles.H"
    #include "sampleParticles.H"

    #include "moveFuelParticles.H"

    runTime.write();
}

Info<< "End mmcMultiphaseFoam \n" << endl;

return 0;
```

Solver – mmcMultiphaseFoam(Momentum)

```
while (runTime.run())
{

    #include "readTimeControls.H"
    #include "compressibleCourantNo.H"
    #include "setDeltaT.H"

    runTime++;
    Info<< "Time = " << runTime.timeName() << nl << en

    #include "rhoEqn.H"

    // --- PIMPLE loop
    while (pimple.loop())
    {
        #include "UEqn.H"

        D = turbulence->nu()/Sc_;
        Dt = turbulence->nut()/Sct_;
        DEff = D + Dt;

        #include "XiEqn.H"
        #include "hYEqvE_Eqn.H"

        // --- Pressure corrector loop
        while (pimple.correct())
        {
            #include "pEqn.H"
        }

        if (pimple.turbCorr())
        {
            turbulence->correct();
        }
    }
```

$$\frac{\partial \bar{\rho}\tilde{u}_i}{\partial t} + \frac{\partial \bar{\rho}\tilde{u}_i\tilde{u}_j}{\partial x_j} = -\frac{\partial \bar{P}}{\partial x_i} + \frac{\partial \tilde{\tau}_{ij}}{\partial x_j} - \frac{\partial \tau_{ij}^{sgs}}{\partial x_j} + \tilde{S}_{u,i}$$

```
fvVectorMatrix UEqn
(
    fvm::ddt(rho, U)
  + fvm::div(phi, U)
  + turbulence->divDevRhoReff(U)
 ==
    rho*g
  + fuelCloud.SuDrag(U)
  + fuelCloud.SuMass(U)
  + fvOptions(rho, U)
);
```

```
        rho = thermo.rho();

        gradRho = fvc::grad(rho*DEff);
        #include "checkParticles.H"    // only works with basicThermoCloud.
        #include "moveParticles.H"
        #include "sampleParticles.H"

        #include "moveFuelParticles.H"

        runTime.write();
    }

    Info<< "End mmcMultiphaseFoam \n" << endl;

    return 0;
```

Solver – mmcMultiphaseFoam (Ref. Mixture fraction)

```
while (runTime.run())
{

    #include "readTimeControls.H"
    #include "compressibleCourantNo.H"
    #include "setDeltaT.H"

    runTime++;
    Info<< "Time = " << runTime.timeName() << nl <<

    #include "rhoEqn.H"

    // --- PIMPLE loop
    while (pimple.loop())
    {
        #include "UEqn.H"

        D = turbulence->nu()/Sc_;
        Dt = turbulence->nut()/Sct_;
        DEff = D + Dt;

        #include "XiEqn.H"
        #include "hYEqvE_Eqn.H"

        // --- Pressure corrector loop
        while (pimple.correct())
        {
            #include "pEqn.H"
        }

        if (pimple.turbCorr())
        {
            turbulence->correct();
        }
    }
```

$$\frac{\partial \bar{\rho}\tilde{f}}{\partial t} + \frac{\partial \bar{\rho}\tilde{u}_i\tilde{f}}{\partial x_i} = -\frac{\partial}{\partial x_i}\left(\tilde{J}_{f,i} + J_{f,i}^{sgs}\right) + \tilde{\dot{S}}_f$$

```
        if (Xi.Vars(XiI).rName() == "f")
        {
            Info<< "Reference Field coupled" << endl;
            fvScalarMatrix XiiEqn
            (
                fvm::ddt(rho,Xii)
              + mvConvectionXi->fvmDiv(phi, Xii)
              - fvm::laplacian(rho * DEff,Xii)
                ==
                rho * (XiiTarget - Xii) / tauRel
              + fuelCloud.SM(Xii)
            );

            XiiEqn.relax();

            if (!pimple.finalIter())
            {
                XiiEqn.solve(mesh.solver("Xii"));
            }
            else
            {
                XiiEqn.solve(mesh.solver("XiiFinal"));
            }


            Xii.max(0.0);

            Xii.min(1.0);

        }
```

```
    }

    In

    return 0;
```

Solver – mmcMultiphaseFoam (Density Coupling)

```
while (runTime.run())
{

    #include "readTimeControls.H"
    #include "compressibleCourantNo.H"
    #include "setDeltaT.H"

    runTime++;
    Info<< "Time = " << runTime.timeName() << nl << endl;

    #include "rhoEqn.H"

    // --- PIMPLE loop
    while (pimple.loop())
    {
        #include "UEqn.H"

        D = turbulence->nu()/Sc_;
        Dt = turbulence->nut()/Sct_;
        DEff = D + Dt;

        #include "XiEqn.H"
        #include "hYEqvE_Eqn.H"

        // --- Pressure corrector loop
        while (pimple.correct())
        {
            #include "pEqn.H"
        }

        if (pimple.turbCorr())
        {
            turbulence->correct();
        }
    }
```

```
fvScalarMatrix hEqvE_Eqn
(
    fvm::ddt(rho, hEqvE)
  + fvm::div(phi, hEqvE)
  - fvm::laplacian(rho * DEff, hEqvE)
 ==
    rho * (hEqvETarget - hEqvE) / tauRel
);
```

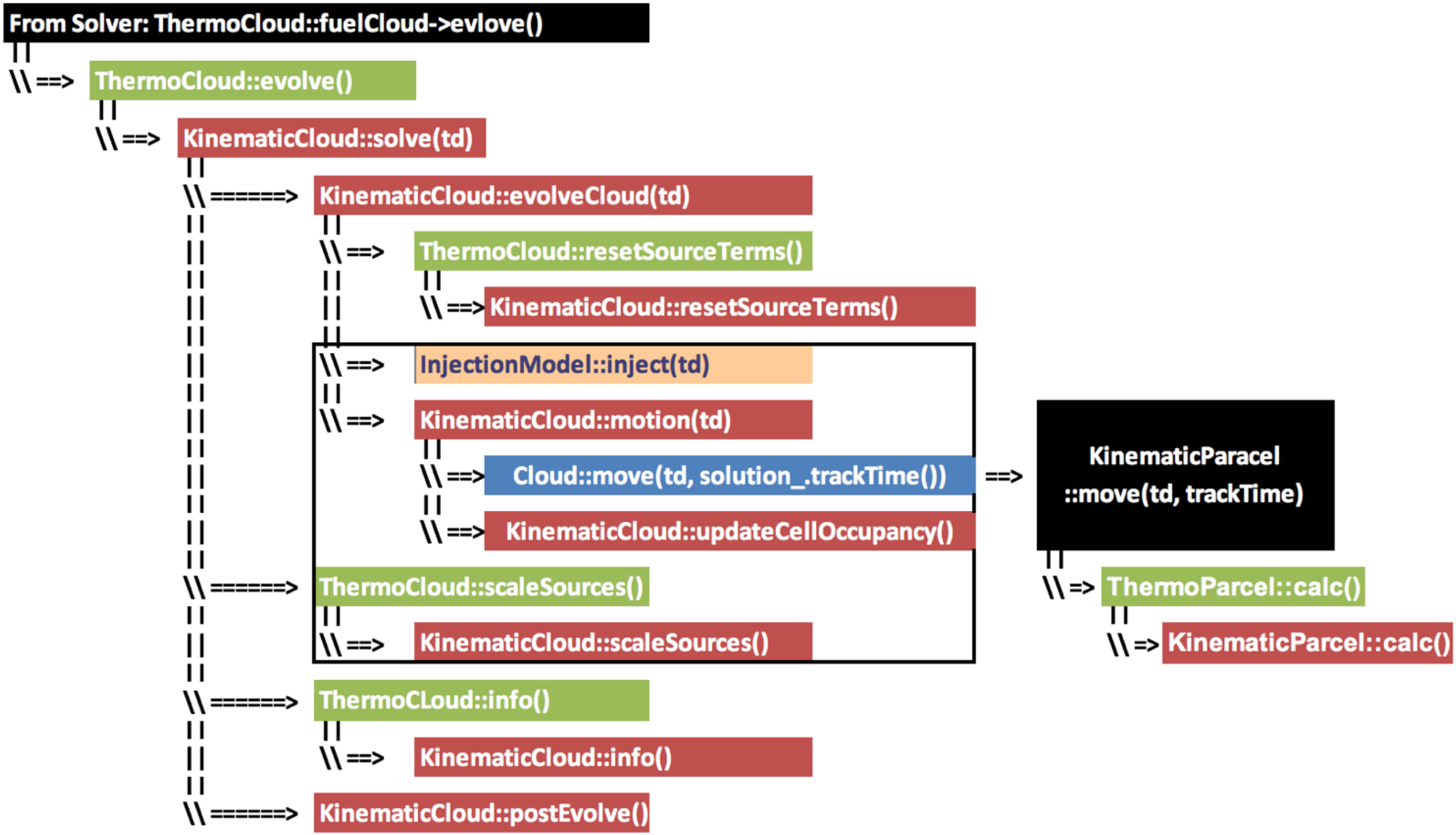
```
fvScalarMatrix YiEqn
(
    fvm::ddt(rho, Yi)
  + fvm::div(phi, Yi)
  - fvm::laplacian(rho * DEff, Yi)
    ==
    rho * (YTarget - Yi) / tauRel
);
```

```
        rho = thermo.rho();

        gradRho = fvc::grad(rho*DEff);
        #include "checkParticles.H"    // only works with basicThermoCloud.
        #include "moveParticles.H"
        #include "sampleParticles.H"

        #include "moveFuelParticles.H"

        runTime.write();
    }

    Info<< "End mmcMultiphaseFoam \n" << endl;

    return 0;
```

Source Code – LFP

– basicThermoCloud road map

```
#include "moveFuelParticles.H"
```



Source Code – LFP

– Implementation of physics in the code

Items in "…" are actual comment in the code

❖ Solid fuel particle calculation

- "Initialize fuel parcel properties"

  Reset *scalarFields* for Gas, Film, Surface, and Middle state mass fractions

- "Initialize source terms"

  Reset momentum and mass source terms for LES solution

- "Bring in cloudProperties dictionary"

  Load the "*cloudProperties*" dictionary

- "Set hash tables for Multiphase - MMC coupling"

- "START calculations"
  - **1. Kinematic calculation**

    Dispersion of solid fuel particle
  - **2. Heat and mass transfer calculation**

    Heat and mass transfer between dispersed phase and carrier phase

Source Code – LFP

– Implementation of physics in the code

❖1. Kinematic calculation

- Location

  ../mmcFoam-4.x/src/lagrangian/multiphase/parcels/Templates/KinematicParcel/KinematicParcel.C

- Implementation (inside KinematicParcel::calc(…))

"obtain the Reynolds number"

"calculate parcel drag coefficient"

"Initialise LES source terms"

"motion of fuel particle"

- calculate parcel drag coefficient.
- calculate parcel relaxation time scale.
- calculate the total acceleration for the fuel parcel.
- update the velocity of the fuel parcel.
- update drag (momentum) source term for the designated cell.

"Accumulate gas phase source terms"

- accumulate the velocity part of the momentum transfer source terms for all LES cells.

Source Code – LFP

– Implementation of physics in the code

❖2. Heat and Mass Transfer calculation

- Location

  ../mmcFoam-4.x/src/lagrangian/multiphase/parcels/Templates/ThermoParcel/ThermoParcel.C

- Implementation (inside ThermoParcel::calc(…))

"Initialize fuel parcel properties"

"Initialize LES source terms"

"Bring in cloudProperties dictionary"

"Set hash tables for Multiphase – MMC coupling" (LFP - MMC)

"Kinematic calculation"

"Thermo calculation"

- Terminate the calculation if the volatile is depleted.
- Update Cp for coal particle based on its volatile fraction and temperature.
- Initialize super cell stochastic particles. And select the one with closest reference mixture fraction for LFP – MMC coupling.
- Reset surface state properties (for coal particles)
- Envelop flame switches (currently left OFF)
- Main iteration loop to determine the surface mixture fraction.
- …

Source Code – LFP

– Implementation of physics in the code

❖2. Heat and Mass Transfer calculation

- Implementation (continued…)

- Main iteration loop to determine the surface mixture fraction.
  - -- Determine surface state mass fraction and enthalpy.
  - -- Determine film state enthalpy and temperature.
  - -- Calculate film state density and viscosity and transport properties.
  - -- Call pyrolysis submodel for surface state mixture fraction.
  - -- Update surface mass fraction and enthalpy.
  - -- Calculate mass flux.
  - -- Finalize the iteration and update surface mixture fraction and mass flux

    If the changes of surface mixture fraction and mass flux between two adjacent iterations are larger than some certain threshold values, further iterations will be required.
  - -- Under relaxation for ThermoParcel iterations.
- Update fuel parcel temperature, mass and diameter.

- Prepare source terms for backward couplings (including LFP – MMC and LFP - LES).
- LFP – LES coupling for mass (momentum) and mixture fraction.
- LFP – MMC coupling.
  - -- …

Source Code – LFP

– Implementation of physics in the code

❖2. Heat and Mass Transfer calculation

- Implementation (continued…)

- LFP – MMC coupling.

-- IF no stochastic particle is found, give birth to a new stochastic particle baring the properties of the volatile mass originated at the surface of the fuel particle. [Commented Out]

-- IF appropriate stochastic particle is found.

-- Update mixture fraction source.

-- Update species mass fraction source.

-- Update enthalpy source.

-- Update stochastic particle mass source.

-- Update stochastic particle properties.

-- Debugging: checking for temperature errors.

--- checking for abnormal stochastic particle temperature.

--- checking for temperature crossing.

--- checking for temperature departing.

- End of LFP calculation

Thank You